

Splatt3R-SLAM: From Image Pairs to Persistent Maps

Real-time photorealistic dense SLAM with 3D Gaussian primitives

Zelong Li

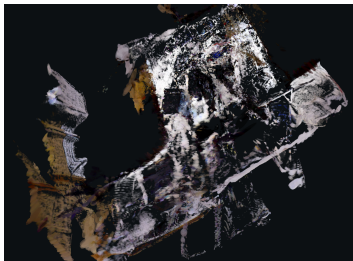
MSc Computer Science — University of Amsterdam

Supervisors: Prof. Martin Oswald Dr. Qi Zhang

October 2026

Talk outline

- ① The problem: an image pair is not a map
- ② The idea: share the anchor
- ③ The method: predict – anchor – refine
- ④ Results on real sequences
- ⑤ Limitations and conclusion



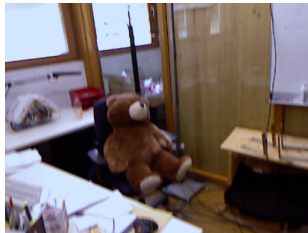
The map as individual Gaussian primitives

An image pair is not a map

One pair gives a useful local result.

But a video sequence adds problems a two-view model never sees:

- every pair lives in its own camera frame;
- many keyframes place colour on one ray;
- a loop closure changes where old content belongs.



Two real input views (TUM)

What a persistent map must survive

Revit

A later camera must still localise.

Correct

Loop closure must move old content.

Render

A new view must look like the scene.

Central requirement

Tracking, pose correction, and rendering must describe the **same** scene.

The thesis: share the anchor

Store the local Gaussian map **and** the cameras that supervise it under the **same keyframe anchor**.

$$T_{WC_f}^{-1} T_{WC_k} = T_{C_k C_f}^{-1}$$

- a pose correction updates one anchor transform;
- the linked map and every observation camera move together;
- their relative relation is preserved exactly.

Method: one shared predictor, two lanes

Geometry lane

pointmaps and matching features feed the tracker and pose graph.

Gaussian lane

one primitive per valid pixel:

$$\Sigma_n = R(q_n) \text{diag}(s_n^2) R(q_n)^\top$$



Real view (left) and predicted map

Refine: use the sequence without moving the tracker

A separate process repeatedly:

- ① samples a tracked view (reservoir + recent ring);
- ② renders the assembled Gaussian map;
- ③ measures $0.8 L_1 + 0.2 \text{ D-SSIM}$;
- ④ takes an Adam step on appearance only.

Gradients never update poses or tracking weights.

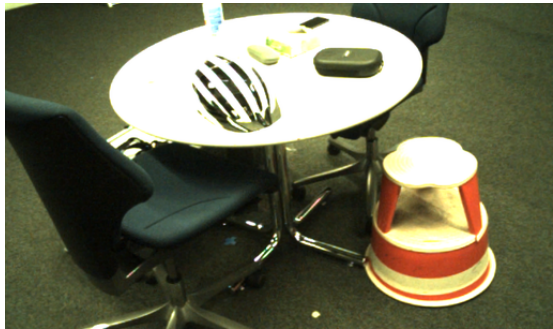


System: concurrent processes

Device	Components	Role
GPU 0	Tracker, Backend, Visualiser	predict, track, pose graph, render
CPU	Supervision frames, Snapshot	sampling pool, versioned map
GPU 1	Refiner	appearance optimisation

- SharedKeyframes exchanged through CUDA IPC;
- supervision and snapshots through CPU shared memory;
- tracking owns poses; refinement only publishes maps.

Results: real sequences



Quantitative comparison (common renderer)

Scene (real)	Photo-SLAM	MonoGS	Ours
TUM desk PSNR (dB)	10.67	13.91	13.95
TUM desk LPIPS (Alex)↓	0.46	0.395	0.411

- real desk: +3.28 dB over Photo-SLAM;
- parity with MonoGS, with slightly higher LPIPS;
- on the synthetic Replica benchmark: 23.02 dB, 8/8 scene wins.

Limitations

- tracking is inherited from MAST3R-SLAM, not a new contribution;
- full maps are large (~ 2.4 M primitives) — not yet compact;
- thin structures and cables carry the most residual blur;
- single runs can hide system variance;
- real-world novel-view benchmarks remain sparse in the literature.

- ▶ **Predict**: dense local Gaussians in one pass;
- ▶ **Anchor**: corrections move map and cameras together;
- ▶ **Refine**: sequence RGB improves appearance downstream.

One idea

Give the local map and its observation cameras the same keyframe anchor.

Backup: related-work positioning

System	Positioning
Photo-SLAM, MonoGS	optimisation-first Gaussian SLAM (CVPR'24)
Splat-SLAM, SplaTAM	global / RGB-D Gaussian maps (2024)
SEGS-SLAM	structure-enhanced, appearance embedding (2025)
Splatt3R-SLAM	prediction-first priors + shared anchors

Backup: primary settings

Field	Value
Input	monocular RGB
Predictor	frozen Splatt3R
Refinement loss	$0.8 L_1 + 0.2 \text{ D-SSIM}$
Optimiser	Adam, appearance only
Evaluation	common non-keyframes, GT Sim(3) alignment, full exported SH
Metrics	PSNR, LPIPS-AlexNet